



Xi'an Jiaotong-Liverpool University

西交利物浦大學

INT305 Machine Learning

Lecture 12

Revision

Siyue Yu

Department Intelligence Science

siyue.yu02@xjtlu.edu.cn

Supervised Learning, Linear Methods for Regression, Optimization

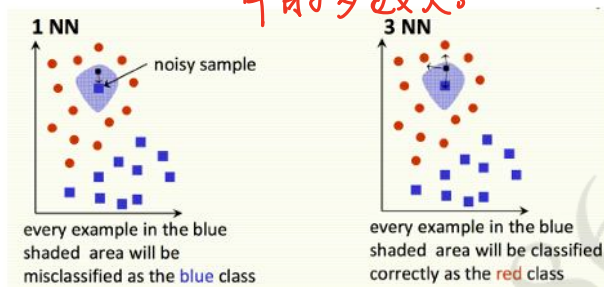
- First learning algorithm: **KNN**
- Second learning algorithm of the course: **linear regression**.
 - **Task**: predict scalar-valued targets (e.g. stock prices)
 - **Architecture**: linear function of the inputs
- While **KNN** was a complete algorithm, linear regression exemplifies a modular approach that will be used through this course:
 - choose a **model** describing the relationships between variables of interest
 - define a **loss function** quantifying how bad the fit to the data is
 - choose a **regularizer** saying how much we prefer different candidate models (or explanations of data)
 - fit a model that minimizes the loss function and satisfies the constraint/penalty imposed by the regularizer, possibly using an **optimization algorithm**
- Mixing and matching these modular components give us a lot of new ML methods.

Lecture 1

k-Nearest Neighbors

找到离 $x^{(a)}$ 最近的 k 个邻居们 $\{x^{(i)}, t^{(i)}\}$. 把 $x^{(a)}$ 分到邻居集合中的多数类。

1. Intro:



ic by Olga Veksler]

用欧氏距离

$$\|x^{(a)} - x^{(b)}\|_2 = \sqrt{\sum_{j=1}^d (x_j^{(a)} - x_j^{(b)})^2}$$

- Nearest neighbors sensitive to noise or mis-labeled data ("class noise"). Solution?
- Smooth by having k nearest neighbors vote.

对噪声敏感

Algorithm (kNN):

1. Find k examples $\{x^{(i)}, t^{(i)}\}$ closest to the test instance x
2. Classification output is majority class

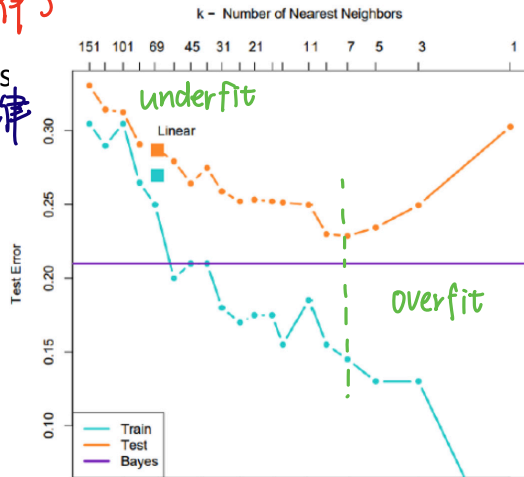
$$y = \arg \max_{t^{(z)}} \sum_{i=1}^k \mathbb{I}(t^{(z)} = t^{(i)})$$

$\mathbb{I}\{\text{statement}\}$ is the identity function and is equal to one whenever the statement is true. We could also write this as $\delta(t^{(z)}, t^{(i)})$, with $\delta(a, b) = 1$ if $a = b$, 0 otherwise.

2. k-Nearest Neighbors 如何选择 k 值 (overfit / underfit)

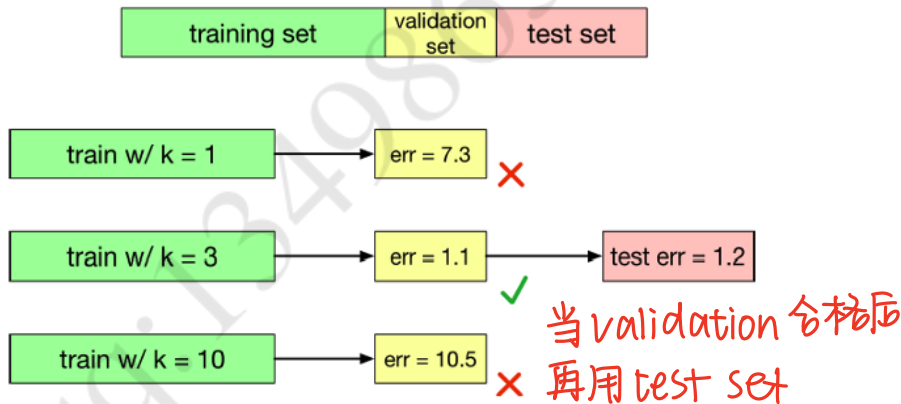
Tradeoffs in choosing k ?

- **Small k** 对于细粒度模式表现良好, 但会过拟合 (sensitive)
 - Good at capturing fine-grained patterns.
 - May **overfit**, i.e., be sensitive to random idiosyncrasies in the training data. 随机特质
- **Large k** 模型较为稳定, 但会 underfit (无法抓住小规律) 过拟合
 - Makes stable predictions by averaging over lots of examples
 - May **underfit**, i.e. fail to capture important regularities. 规律
- **Balancing k**
 - Optimal choice of k depends on number of data points n .
 - Nice theoretical properties if $k \rightarrow \infty$ and $\frac{k}{n} \rightarrow 0$.
 - Rule of thumb: choose $k < \sqrt{n}$.
 - We can choose k using validation set (next slides).



k-Nearest Neighbors 用 validation set 和 test set 来调整超参 k

- k is an example of a hyperparameter, something we can't fit as part of the learning algorithms itself. k 是提前设置的参数, 不是机器学习的一部分
- We can tune hyperparameters using a validation set:



- The test set is used only at the very end, to measure the generalization performance of the final configuration.

Lecture 2

Linear Regression - Model

1. Intro

- **Model**: In linear regression, we use a linear function of the features $\mathbf{x} = (x_1, \dots, x_D) \in \mathbb{R}^D$ to make predictions y of the target value $t \in \mathbb{R}$:

output

$$y = f(\mathbf{x}) = \sum_j w_j x_j + b$$

Input

- y is the **prediction**
 - $\mathbf{w}$ is the **weights**
 - b is the **bias** (or **intercept**)
- $\mathbf{w}$ and b together are the **parameters**
 - We hope that our prediction is close to the target: $y \approx t$.

向量形式: $y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} + b$

权重矩阵

Linear Regression - Loss Function

2. loss function 和 cost function

- A **loss function** $\mathcal{L}(y, t)$ defines how bad it is if, for some example $\mathbf{x}$, the algorithm predicts y , but the target is actually t .

- Squared error loss function:** 衡量一个 example

$$\mathcal{L}(y, t) = \frac{1}{2} (y - t)^2$$

目标是和预测之差 尽可能的

- $y - t$ is the residual (残差), and we want to make this small in magnitude.

- The $\frac{1}{2}$ factor is just to make the calculations convenient.

- Cost function:** loss function averaged over all training examples.

整个训练集

$$\begin{aligned} J(\mathbf{w}, b) &= \frac{1}{2N} \sum_{i=1}^N (y^{(i)} - t^{(i)})^2 = \frac{\sum \mathcal{L}(y^{(i)}, t^{(i)})}{N} \\ &= \frac{1}{2N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} + b - t^{(i)})^2 \end{aligned}$$

训练集中的数据个数

- Terminology varies. Some call “cost” *empirical* or *average loss*.

3. Vectorization 将所有变量转化成一个矩阵

- Computing the squared error cost across the whole dataset:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b\mathbf{1}$$

$$\mathcal{J} = \frac{1}{2N} \|\mathbf{y} - \mathbf{t}\|^2$$

均是 vector. $\vec{y} = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{pmatrix}$

- Sometimes we may use $\mathcal{J} = \frac{1}{2} \|\mathbf{y} - \mathbf{t}\|^2$, without a normalizer. This would correspond to the sum of losses, and not the averaged loss. The minimizer does not depend on N (but optimization might!).
- We can also add a column of 1's to design matrix, combine the bias and the weights, and conveniently write

$$\mathbf{X} = \begin{bmatrix} 1 & [\mathbf{x}^{(1)}]^T \\ 1 & [\mathbf{x}^{(2)}]^T \\ \vdots & \vdots \end{bmatrix} \in \mathbb{R}^{N \times (D+1)} \quad \text{and} \quad \mathbf{w} = \begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \end{bmatrix} \in \mathbb{R}^{D+1}$$

N 个 example, D 个 feature
1 个截距

D 个 feature, 1 个截距

Then, our predictions reduce to $\mathbf{y} = \mathbf{X}\mathbf{w}$.

4. min loss function

Direct Solution II: Calculus

为 min loss function. 对两个参数 w_j 和 b 分别求偏导, 逼近 Critical point (导 = 0)

- For loss derivatives, apply the **chain rule**:

原式形式

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w_j} &= \frac{d\mathcal{L}}{dy} \frac{\partial y}{\partial w_j} \\ &= \frac{d}{dy} \left[\frac{1}{2} (y - t)^2 \right] \cdot x_j \\ &= (y - t) x_j \quad \textcircled{1} \times \textcircled{3}\end{aligned}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial b} &= \frac{d\mathcal{L}}{dy} \frac{\partial y}{\partial b} \\ &= y - t \\ &\quad \textcircled{2} \times \textcircled{3}\end{aligned}$$

$\mathcal{L}(y, t) = \frac{1}{2} (y - t)^2$ 过程

$y = \sum_j w_j x_j + b$

$\textcircled{1} \frac{\partial y}{\partial w_j} = x_j$ $\textcircled{2} \frac{\partial \mathcal{L}}{\partial b} = 1$

$\textcircled{3} \frac{\partial \mathcal{L}}{\partial y} = y - t$

- For cost derivatives, use **linearity** and **average over data points**:

矩阵形式

$$\frac{\partial J}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)})$$

$J = \frac{1}{2N} \|y - t\|^2$

$\frac{\partial J}{\partial w_j} = \frac{1}{N} \sum (y^{(i)} - t^{(i)}) x_j^{(i)}$

- Minimum must occur at a point where partial derivatives are zero.

偏导 = 0, minimum 发生

$$\frac{\partial J}{\partial w_j} = 0 \quad (\forall j),$$

$$\frac{\partial J}{\partial b} = 0.$$

$\frac{\partial J}{\partial w_j} = \frac{\partial J}{\partial y} \cdot \frac{\partial y}{\partial w_j} = \frac{1}{N} \sum (y^{(i)} - t^{(i)}) x_j^{(i)}$

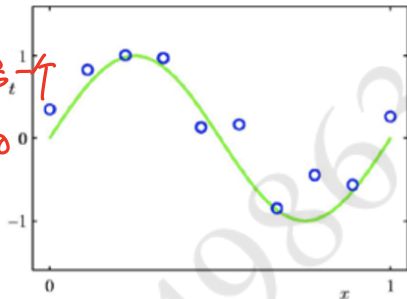
(If $\partial J / \partial w_j \neq 0$, you could reduce the cost by changing w_j)

result: $w^* = (X^T X)^{-1} X^T t$

5. Polynomial Feature Mapping 多项式特征映射

The relation between the input and output may not be linear.

对于非线性输入: 映射到另一个空间 $\mathbb{R}^D \rightarrow \mathbb{R}^d$, 再将 $\mathbb{R}^d$ 中的作为 input 进行线性回归。



- We can still use linear regression by mapping the input features to another space using **feature mapping (or basis expansion)**. $\varphi(\mathbf{x}): \mathbb{R}^D \rightarrow \mathbb{R}^d$ and treat the mapped feature (in $\mathbb{R}^d$) as the input of a linear regression procedure.
- Let us see how it works when $\mathbf{x} \in \mathbb{R}$ and we use a polynomial feature mapping.

Find the data using a degree-M polynomial function of the form:

$$y = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{i=0}^M w_i x^i$$

- Here the feature mapping is $\varphi(x) = [1, x, x^2, \dots, x^M]^T$.
- We can use linear regression to find $\mathbf{w}$, since $y = \varphi(x)^T \mathbf{w}$ is linear with $w_0, w_1, \dots$

M过大: overfit

M过小: underfit

M: complexity (复杂度)

6. L2 Regularization 惩罚过多参数, 防止过拟合

- We can encourage the weights to be small by choosing as our regularizer the L^2 penalty. 惩罚 指示是否 overfit $\mathcal{R}(\mathbf{w}) = \frac{1}{2} \sum_j w_j^2$.

➤ Note: To be precise, the L^2 norm is Euclidean distance, so we're regularizing the squared L^2 norm.

- The regularized cost function makes a tradeoff between fit to the data and the norm of the weights. L^2 范数是欧氏距离 $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{t}\|^2$ cost function

$$J_{reg}(\mathbf{w}) = J(\mathbf{w}) + \lambda \mathcal{R}(\mathbf{w}) = J(\mathbf{w}) + \frac{\lambda}{2} \sum_j w_j^2$$

cost is large.

- If you fit training data poorly, J is large. If your optimal weights have high values, $\mathcal{R}$ is large.
- Large λ penalizes weight values more.
- Like M , λ is a hyperparameter we can tune with a validation set.

7. Gradient Descent

- Observe:

- If $\partial J / \partial w_j > 0$, then increasing w_j increases J . 导数 > 0, 递增
- If $\partial J / \partial w_j < 0$, then increasing w_j decreases J . < 0, 递减

- The following update always decreases the cost function for small enough α (unless

③ $\partial J / \partial w_j = 0$):

Gradient:

$$\nabla_w J = \frac{\partial J}{\partial w} = \begin{pmatrix} \frac{\partial J}{\partial w_1} \\ \vdots \\ \frac{\partial J}{\partial w_p} \end{pmatrix}$$

①

learning rate

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}$$

②

代入线性回归:

$$W \leftarrow W - \frac{\alpha}{N} \sum (y^{(i)} - t^{(i)}) x^{(i)}$$

- $\alpha > 0$ is a learning rate (or step size). The larger it is, the faster w changes.

④ update to minimize J :

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}$$

⑤

结合 6. L2 Regularization: cost $J = J + \lambda R$

$$w \leftarrow (1 - \alpha \lambda) w - \alpha \frac{\partial J}{\partial w}$$

8 Stochastic Gradient Descent 随机梯度下降

- Stochastic gradient descent (SGD): update the parameters based on the gradient for a single training example,

average loss:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}^{(i)}$$

$$= \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y(x^{(i)}, \theta), t^{(i)})$$

$$\theta = (w, b) \quad 2- \theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial \theta}$$

$$\frac{\partial J}{\partial \theta} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}^{(i)}}{\partial \theta}$$

- Cost of each SGD update is independent of N !

- SGD can make significant progress before even seeing all the data!
- Mathematical justification: if you sample a training example uniformly at random, the

stochastic gradient is an unbiased estimate of the batch gradient:

$$\mathbb{E} \left[\frac{\partial \mathcal{L}^{(i)}}{\partial \theta} \right] = \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}^{(i)}}{\partial \theta} = \frac{\partial J}{\partial \theta}$$

如果你对-个训练集
样本均匀随机取样，
那么随机梯度就是批梯度的
无偏估计。

Stochastic Gradient Descent

- Problems with using single training example to estimate gradient:

- Variance in the estimate may be high
- We can't exploit efficient vectorized operations

- Compromise approach:

mini-batch: 选择进行梯度下降的样本.

- Compute the gradients on a randomly chosen medium-sized set of training examples $\mathcal{M} \subset \{1, \dots, N\}$, called a mini-batch.

larger $|\mathcal{M}|$, smaller variance

- Stochastic gradients computed on larger mini-batches have smaller variance.
- The mini-batch size $|\mathcal{M}|$ is a hyperparameter that needs to be set.
 - Too large: requires more compute: e.g., it takes more memory to store the activations, and longer to compute each gradient update
 - Too small: can't exploit vectorization, has high variance
 - A reasonable value might be $|\mathcal{M}| = 100$.

对于 learning rate: 早期用大的, 后期逐步下降.

Linear Classifiers, Logistic Regression, Multiclass Classification

- **Classification**: predicting a discrete-valued target
 - **Binary classification**: predicting a binary-valued target
 - **Multiclass classification**: predicting a discrete(>2)-valued target
- Examples of binary classification
 - Predict whether a patient has a disease, given the presence or absence of various symptoms
 - Classify e-mails as spam or non-spam
 - Predict whether a financial transaction is fraudulent

Lecture 3

利用linear function和threshold
判断 0/1 问题

1. Binary linear classification

1. Intro

- **Classification:** given a D -dimensional input $\mathbf{x} \in \mathbb{R}^D$ predict a discrete-valued target
- **Binary:** predict a binary target $t \in \{0, 1\}$
 - Training examples with $t = 1$ are called **positive examples**, and training examples with $t = 0$ are called **negative examples**.
 - $t \in \{0, 1\}$ or $t \in \{-1, +1\}$ is for computational convenience.
- **Linear:** model prediction y is a linear function of $\mathbf{x}$, followed by a threshold r :

预测结果 $z = \mathbf{w}^T \mathbf{x} + b$ 先计算 linear 值. 判断标准

$$y = \begin{cases} 1 & \text{if } z \geq r \\ 0 & \text{if } z < r \end{cases}$$

再根据 z 的大小,
利用阈值判断正/负

Simplifications

利用threshold, 转换成0/1问题

2. Eliminating the threshold

不损失一般性

- We can assume without loss of generality (WLOG) that the threshold $r = 0$:

$$\mathbf{w}^T \mathbf{x} + b \geq r \iff \underbrace{\mathbf{w}^T \mathbf{x} + b - r}_{\triangleq w_0} \geq 0$$

Eliminating the bias

虚拟特征

- Add a dummy feature x_0 which always takes the value 1. the weight $w_0 = b$ is equivalent to a bias (same as linear regression)

Simplified model

We can also add a column of 1's to design matrix, combine the bias and the weights, and conveniently write

- Receive input $\mathbf{x} \in \mathbb{R}^{D+1}$ with $x_0 = 1$:

$$\mathbf{X} = \begin{bmatrix} 1 & [\mathbf{x}^{(1)}]^T \\ 1 & [\mathbf{x}^{(2)}]^T \\ \vdots & \vdots \end{bmatrix} \in \mathbb{R}^{N \times (D+1)} \quad \text{and} \quad \mathbf{w} = \begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \end{bmatrix} \in \mathbb{R}^{D+1}$$

横过去乘矩阵X

$$Z = \mathbf{w}^T \mathbf{x}$$

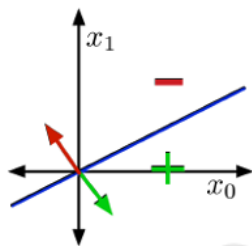
Then, our predictions reduce to $\mathbf{y} = \mathbf{X}\mathbf{w}$.

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

dummy feature x_0

3. The Geometric Picture 图像表示

Input Space, or Data Space for NOT example



x_0	x_1	t
1	0	1
1	1	0

满足所有条件的 region 为 feasible region.
若该区域非空, 则 problem feasible.

- Training examples are points
- Weights (hypotheses) $\mathbf{w}$ can be represented by half-spaces

$$H_+ = \{\mathbf{x}: \mathbf{w}^\top \mathbf{x} \geq 0\}, H_- = \{\mathbf{x}: \mathbf{w}^\top \mathbf{x} < 0\}$$

所有参数均为0时, 过原点

➤ The boundaries of these half-spaces pass through the origin (why?)

- The boundary is the decision boundary: $\{\mathbf{x}: \mathbf{w}^\top \mathbf{x} = 0\}$

➤ In 2-D, it's a line, but in high dimensions it is a hyperplane

- If the training examples can be perfectly separated by a linear decision rule, we say data is linearly separable.

Summary | Binary Linear Classifiers

- **Summary:** targets $t \in \{0,1\}$, inputs $\mathbf{x} \in \mathbb{R}^{D+1}$ with $x_0 = 1$, and model is defined by weights $\mathbf{w}$ and

$$z = \mathbf{w}^\top \mathbf{x}$$

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

- How can we find good values for $\mathbf{w}$?
- If training set is linearly separable, we could solve for $\mathbf{w}$ using linear programming
 - We could also apply an iterative procedure known as the perceptron algorithm (but this is primarily of historical interest).
- If it's not linearly separable, the problem is harder
 - Data is almost never linearly separable in real life.

感知计算法

4. loss function: logistic Activation function

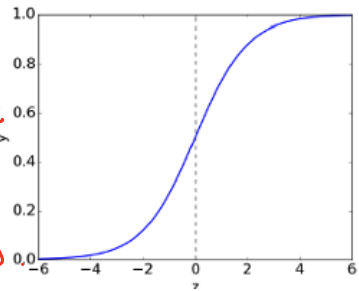
通过压缩 z , 使误差不会较大正、较小负而靠近0

- There's obviously no reason to predict values outside $[0, 1]$. Let's squash y into this interval. S型函数

- The **logistic function** is a kind of **sigmoid**, or S-shaped function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$z \leq 0, 0 < \sigma(z) \leq \frac{1}{2}$
预测为 negative (0)
 $z > 0, \frac{1}{2} < \sigma(z) < 1$
预测为 positive (1)



- $\sigma^{-1}(y) = \log(y/(1 - y))$ is called the **logit**.
- A linear model with a logistic nonlinearity is known as **log-linear**:

$$z = \mathbf{w}^T \mathbf{x}$$

$$y = \sigma(z)$$

将线性预测结果 z sigmoid化得到 y 再和正确结果 t 比较

loss function: $\mathcal{L}_{SE}(y, t) = \frac{1}{2} (y - t)^2$

- Used in this way, σ is called an **activation function**. 激活函数
可以衡量measure的确定性和实际值之间的差距。

5. Logistic Regression : Cross - entropy loss

- Because $y \in [0, 1]$, we can interpret it as the estimated probability that $t = 1$. If $t = 0$, then we want to heavily penalize $y \approx 1$. 重罰
- The pundits who were 99% confident Clinton would win were much more wrong than the ones who were only 90% confident.
- Cross-entropy loss** (aka log loss) captures this intuition:

交叉熵损失

$$\mathcal{L}_{CE}(y, t) = \begin{cases} -\log y & \text{if } t = 1 \\ -\log(1 - y) & \text{if } t = 0 \end{cases}$$

y 已 activation

$$= -t \log y - (1 - t) \log(1 - y)$$

1. $t=0$ 时

$$\mathcal{L}_{CE}(y, t) = -\log(1 - y)$$

① $y \rightarrow 0$ 时, $1 - y \rightarrow 1$, $\mathcal{L}_{CE}(y, t) \rightarrow 0$ loss 小

② $y \rightarrow 1$ 时, $1 - y \rightarrow 0$, $\mathcal{L}_{CE}(y, t) \rightarrow \infty$ loss 大

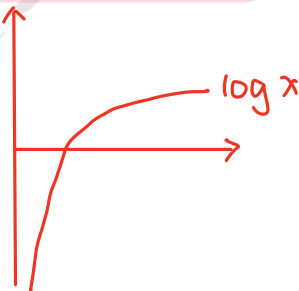
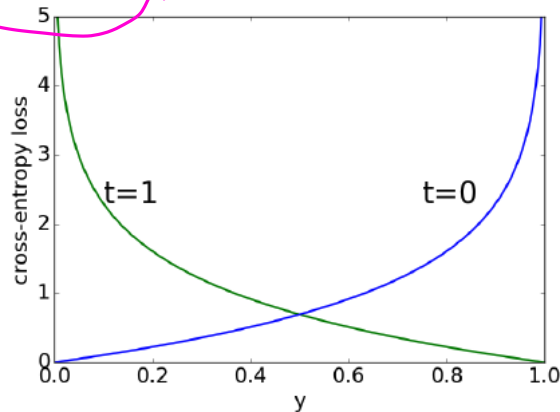
2. $t=1$ 时, $\mathcal{L}_{CE}(y, t) = -\log y$

① $y \rightarrow 0$ 时, $\mathcal{L}_{CE}(y, t) \rightarrow \infty$ loss 大

② $y \rightarrow 1$ 时, $\mathcal{L}_{CE}(y, t) \rightarrow 0$ loss 小

$$z = W^T x$$

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$



6. Gradient Descent for Logistic Regression

Comparison of gradient descent updates:

- Linear regression:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$$

- Logistic regression:

learning rate

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$$

- Not a coincidence! These are both examples of generalized linear models. But we won't go in further detail.
- Notice $\frac{1}{N}$ in front of sums due to averaged losses. This is why you need smaller learning rate when cost is summed losses ($\alpha' = \alpha/N$).

Back to logistic regression:

$$\mathcal{L}_{\text{CE}}(y, t) = -t \log y - (1 - t) \log(1 - y)$$

$$y = 1/(1 + e^{-z}) \text{ and } z = \mathbf{w}^T \mathbf{x}$$

Therefore

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{CE}}}{\partial w_j} &= \frac{\partial \mathcal{L}_{\text{CE}}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_j} = \left(-\frac{t}{y} + \frac{1-t}{1-y} \right) \cdot y(1-y) \cdot x_j \\ &= (y - t)x_j \end{aligned}$$

(verify this)

Gradient descent (coordinate-wise) update to find the weights of logistic regression:

$$\begin{aligned} w_j &\leftarrow w_j - \alpha \frac{\partial \mathcal{J}}{\partial w_j} \\ &= w_j - \alpha \sum_{i=1}^N (y^{(i)} - t^{(i)}) x_j^{(i)} \end{aligned}$$

广义线性模型

★ α 是针对单example的loss. α' 是针对变量矩阵 $\mathbf{X}$ 的loss

Multiclass Linear Classification

输出的output有多个维度 $z = \{z_1, \dots, z_K\}$

1. Intro:

- We can start with a linear function of the inputs. 有K个不同的维度
- Now there are D input dimensions and K output dimensions, so we need $K \times D$ weights, which we arrange as a weight matrix W .
- Also, we have a K -dimensional vector b of biases.
- A linear function of the inputs:

$$z_k = \sum_{j=1}^D w_{kj} x_j + b_k \quad \text{for } k = 1, 2, \dots, K$$

不同的input, 不同的
维度, 均需要不同
的weight

- We can eliminate the bias b by taking $W \in \mathbb{R}^{K \times (D+1)}$ and adding a dummy variable $x_0 = 1$. So, vectorized: 把b合并进W, 添加个dummy x0

$$z = Wx + b \quad \text{or with dummy } x_0 = 1 \quad z = Wx$$

$$z \in \mathbb{R}^K, x \in \mathbb{R}^D, W \in \mathbb{R}^{K \times D}, b \in \mathbb{R}^K$$

$$z \in \mathbb{R}^K, x \in \mathbb{R}^D, W \in \mathbb{R}^{K \times (D+1)}$$

2. Multiclass Linear Classification : One-hot prediction

分类结果为有 K 个维度的向量. 预测哪个 class k_i , k_i 在向量中的对应位置为 1. 其余为 0.

- How can we turn this linear prediction into a **one-hot prediction**?
- We can interpret the magnitude of z_k as a measure of how much the model prefers k as its prediction. 量级
- If we do this, we should set

$$y_i = \begin{cases} 1 & i = \arg \max_k z_k \\ 0 & \text{otherwise} \end{cases}$$

分类的可能性最大的类

- Targets form a discrete set $\{1, \dots, K\}$. 属于哪一类, 它的哪个位置为 1
- It's often more convenient to represent them as **one-hot vectors**, or a **one-of-K encoding**:

编码

$$\mathbf{t} = (0, \dots, 0, 1, 0 \dots, 0) \in \mathbb{R}^K$$

entry k is 1

3. Softmax Regression

Softmax: soft predictions as probabilities

- Softmax regression (with dummy $x_0 = 1$):

$$y_k = \frac{e^{z_k}}{\sum_{k'} e^{z_{k'}}}$$

类 k 的 e^{z_k}
所有类别的 e^{z_k} 之和

softmax-cross-entropy $\mathbf{z} = \mathbf{W}\mathbf{x}$

$$\mathbf{y} = \text{softmax}(\mathbf{z})$$

$$\mathcal{L}_{\text{CE}} = -\mathbf{t}^T(\log \mathbf{y}) = -\sum_k t_k \log y_k$$

- Gradient descent updates can be derived for each row of $\mathbf{W}$:

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial \mathbf{w}_k} = \frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_k} \cdot \frac{\partial z_k}{\partial \mathbf{w}_k} = (y_k - t_k) \cdot \mathbf{x}$$

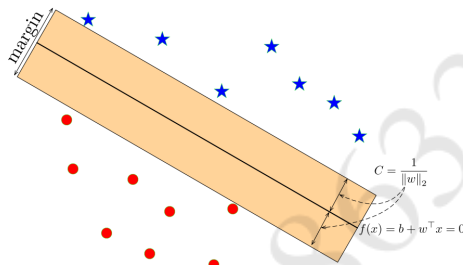
$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \alpha \frac{1}{N} \sum_{i=1}^N (y_k^{(i)} - t_k^{(i)}) \mathbf{x}^{(i)}$$

- Similar to linear/logistic reg (no coincidence) (verify the update)

Lecture 4

- SVM

1. Intro



点 x' 到 hyperplane 的距离

$$\frac{w^T x' + b}{\|w\|_2}$$

A **Support Vector Machine (SVM)** is a supervised machine learning algorithm used for classification and regression tasks. It is particularly effective in high-dimensional spaces and is well-suited for scenarios where there is a clear margin of separation between classes. 当有清晰界线时, 效果好

The basic idea behind SVM is to find the hyperplane that **best divides a dataset into two classes**.

The term "**support vector**" refers to the data points that lie closest to the decision boundary, and these points are crucial in determining the optimal hyperplane.

margin: distance to the closest data points from either classes

The **goal** is to maximize the margin, which is the distance between the hyperplane and the nearest data points of each class.

优化问题: max margin (不考虑松弛也是, Hard)
2. Maximizing Margin as an Optimization Problem **Hard margin**

目标

Max-margin objective:

$$\max_{\mathbf{w}, b} C$$

$t^{(i)}: x^{(i)}$ 的正确 label

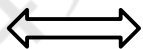
$$\text{s.t. } \frac{t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2} \geq C \quad i = 1, \dots, N$$

代入

Plug in $C = 1/\|\mathbf{w}\|_2$ and simplify:

$$\frac{t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2} \geq \frac{1}{\|\mathbf{w}\|_2}$$

geometric margin constraint



$$t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1$$

algebraic margin constraint

Equivalent optimization objective:

$$\min \|\mathbf{w}\|_2^2$$

满足该等式的最小 $\|\mathbf{w}\|_2^2$

$$\text{s.t. } t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 \quad i = 1, \dots, N$$

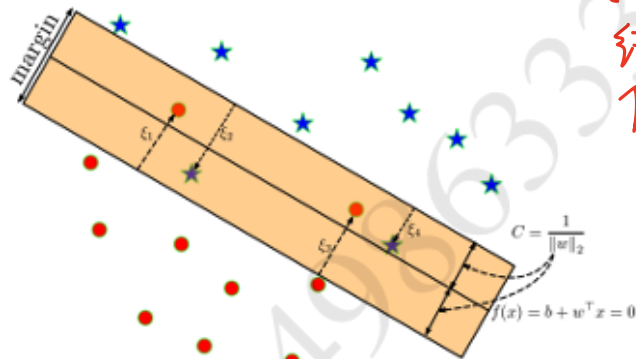
subject to 满足

The important training examples are the ones with algebraic margin 1, and are called support vectors.

优化问题: 考虑松弛变量. Soft

3. Maximizing Margin for Non-Separable Data Points 引入 slack variables

Soft-margin SVM



(松弛变量) ξ_i . 解决非完全线性可分.
但限制或惩罚 ξ_i

- **Soft margin constraint:**

$$\frac{t^{(i)}(w^T x^{(i)} + b)}{\|w\|_2} \geq C(1 - \xi_i),$$

for $\xi_i \geq 0$. slack variable (misclassified / within the margin)

- Penalize $\sum_i \xi_i$.

Maximizing Margin for Non-Separable Data Points

Soft-margin SVM objective:

Soft margin

$$\min_{\mathbf{w}, b, \xi} \frac{C}{2} \|\mathbf{w}\|_2^2 + \gamma \sum_{i=1}^N \xi_i$$

C (margin) γ 松弛系数 ①

subject to 满足

$$\text{s.t. } t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \xi_i \quad i = 1, \dots, N$$
$$\xi_i \geq 0 \quad i = 1, \dots, N$$

- γ is a hyperparameter that trades off the margin with the amount of slack.
 - For $\gamma = 0$, we'll get $\mathbf{w} = 0$. (why?) $\sum \xi_i = 0$. $\mathbf{w}$ 取 0 时, 原式 min.
 - As $\gamma \rightarrow \infty$, we get the hard-margin objective.
- Note: it is also possible to constrain $\sum_i \xi_i$ instead of penalizing it.

$$\xi_i = \max\{0, 1 - t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)\}$$

令 $y^{(i)}(\mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b$. Optimization 为

loss function $\mathcal{L}_H(y, t) = \max\{0, 1 - ty\}$

$$\min_{\mathbf{w}, b, \xi} \sum_{i=1}^N \max\{0, 1 - t^{(i)} y^{(i)}(\mathbf{w}, b)\} + \frac{1}{2\gamma} \|\mathbf{w}\|_2^2$$

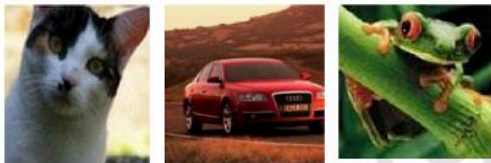
(等式 1 除 γ)

二. loss的计算

1. multiclass SVM loss

Suppose: 3 training examples, 3 classes.

With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

$$\max\{0, (5.1 - 3.2)\} \\ + \max\{0, (-1.7 - (-3.1))\} = 2.9$$

不考虑概率, 只考虑绝对分数.

Multiclass SVM loss:

Given an example (x_i, y_i) ,
where x_i is the image and y_i is the
(integer) label.

and using the shorthand for the scores
vector: $s = f(x_i, W)$

s_j : 非class类的得分

The SVM loss has the form: s_{y_i} : class类的得分

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

and the full training loss is the mean over
all examples in the training data:

$$L = \frac{1}{N} \sum_{i=1}^N L_i \\ L = (2.9 + 0 + 12.9)/3 \\ = 5.27$$

2. Softmax

考虑分类正确的概率.

Softmax Classifier (Multinomial Logistic Regression)



对右图, $L_i = -\log\left(\frac{e^{3.2}}{e^{3.2} + e^{5.1} + e^{-1.7}}\right)$
 $= -\log\left(\frac{24.5}{24.5 + 164 + 0.18}\right)$ cat
 $= -\log(0.13)$ car
 $= 0.89$ frog



3.2

5.1

-1.7

已知变量, 分类正确的概率为: 最大化 P

scores = unnormalized log probabilities of the classes.

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}} \text{ where } s = f(x_i; W)$$

s_k : 目标类的预测分数 s_j : 所有类的预测分数.

Want to maximize the log likelihood, or (for a loss function) to minimize the negative log likelihood of the correct class:

最小化 L_i

$$L_i = -\log P(Y = y_i | X = x_i) = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

使 $P \rightarrow 1$, $\frac{e^{s_k}}{\sum_j e^{s_j}} \rightarrow 1$, 则使 $e^{s_k} \uparrow$ or/and 使 $\sum_j e^{s_j} \downarrow$

SVM & Softmax

Softmax vs. SVM

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

s_{y_i} : 正确class的得分

s_j : 所有class的得分.

L_i 为损失, 则 negative
log-分对的 likelihood.

要使 L_i min.

考虑了可能性

multiclass SVM loss

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

s_j : 错误class的得分

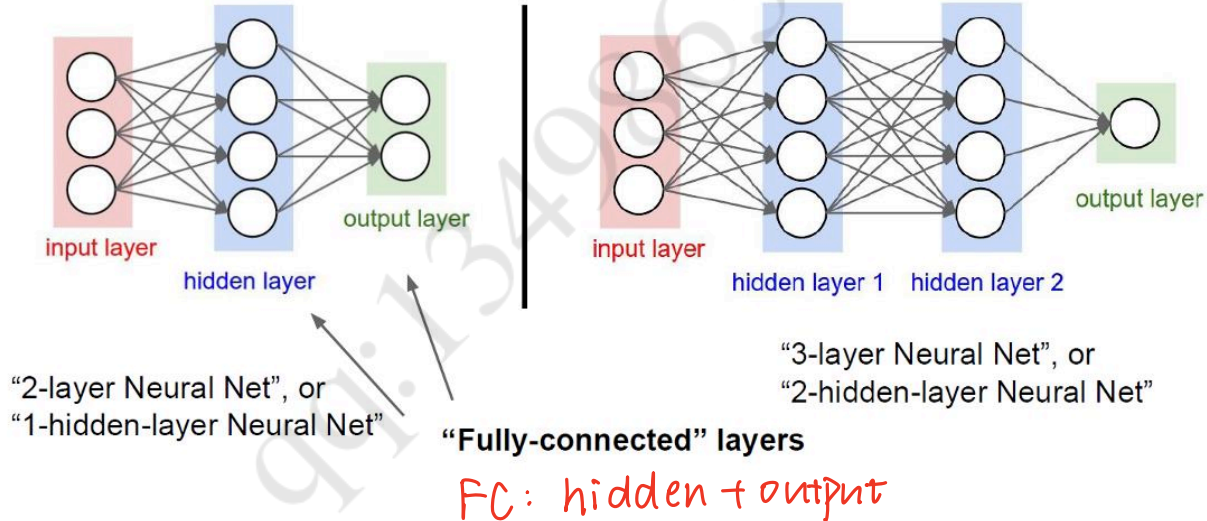
s_{y_i} : 正确class的得分.

无normalize (取 exp), 会受到分数等级的影响
不考虑可能性.

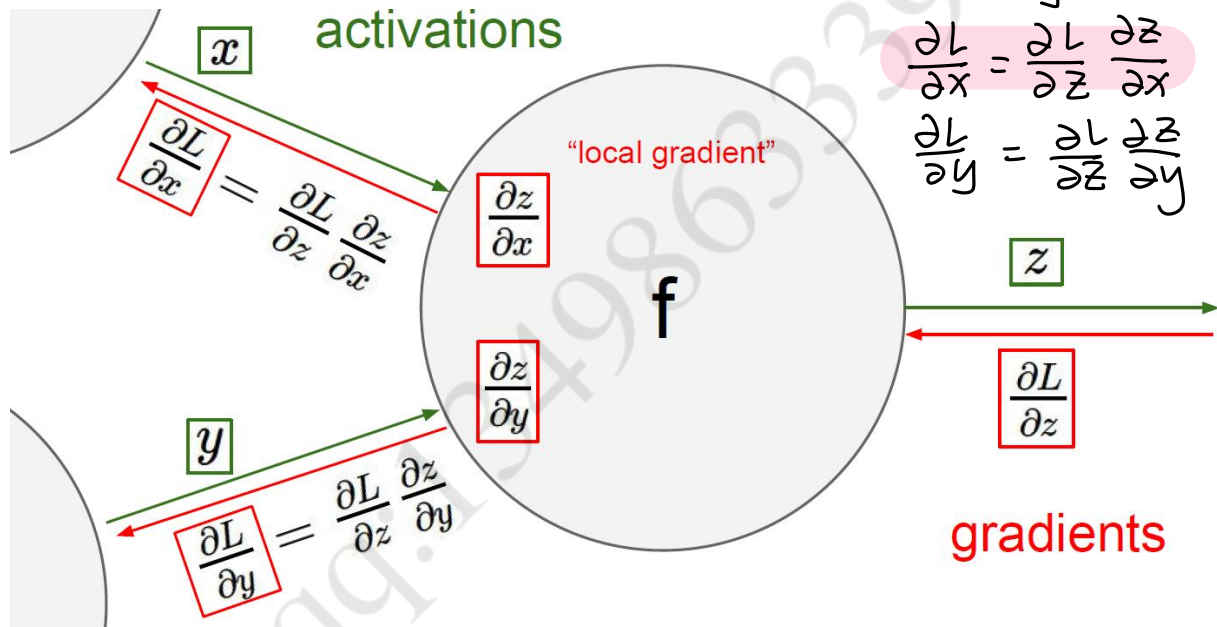
Lecture 5 - Neural network

Input ($\rightarrow$ activation) $\rightarrow$ hidden layer ($\times n$) $\rightarrow$ output.

Neural Networks: Architectures



Chain rule 1. 基本原理



Chain rule (Back Propagation) 2. 代入数值一步步求导

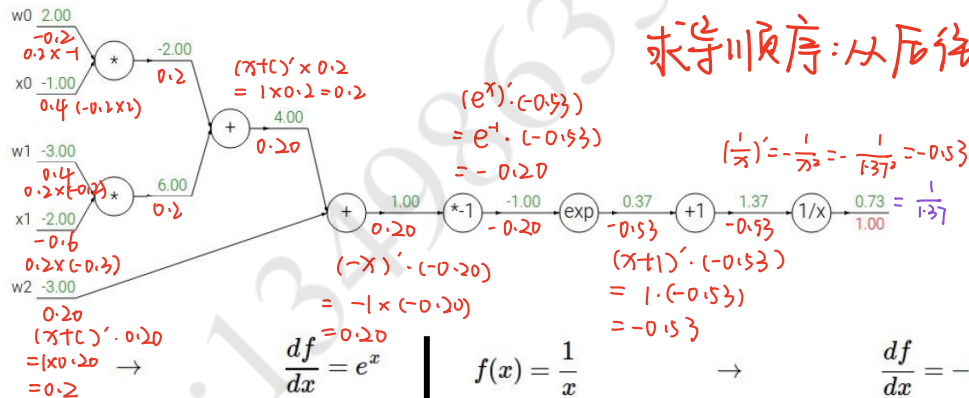
计算方式: local gradient $\times$ its gradient

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$$

= 对当前步骤求导 $\times$ 之前求导结果

求导顺序: 从后往前 (右 $\rightarrow$ 左).



$$f(x) = e^x$$

$$\frac{df}{dx} = e^x$$

$$f(x) = \frac{1}{x}$$

$$\frac{df}{dx} = -1/x^2$$

Sigmoid gate: $f_a(x) = ax$

$$\frac{df}{dx} = a$$

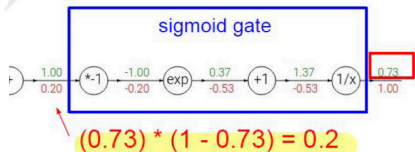
$$f_c(x) = c + x$$

$$\frac{df}{dx} = 1$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

sigmoid function

$$\sigma'(x) = x \cdot (1 - x)$$



$$(0.73) \cdot (1 - 0.73) = 0.2$$

Gradients for vector 3. 向量的求导.

A vectorized

$$\begin{bmatrix} W_{1,1} & W_{1,2} \\ W_{2,1} & W_{2,2} \end{bmatrix} = \begin{bmatrix} 0.1 & 0.5 \\ -0.3 & 0.8 \end{bmatrix}$$

$$\begin{bmatrix} 0.088 & 0.176 \\ 0.104 & 0.208 \end{bmatrix} W$$

$$\begin{bmatrix} -0.112 \\ 0.636 \end{bmatrix}$$

$$\begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix} \begin{matrix} x_1 \\ x_2 \end{matrix}$$

$$\frac{\partial q_k}{\partial x_i} = W_{k,i}$$

$$\frac{\partial f}{\partial x_i} = \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial x_i}$$

$$= \sum_k 2q_k W_{k,i}$$

$$\frac{\partial q_k}{\partial x_i} = W_{k,i}$$

$$\frac{\partial f}{\partial x_i} = \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial x_i}$$

$$= \sum_k 2q_k W_{k,i}$$

$$\|W \cdot x\|^2 = \sum_{i=1}^n (W \cdot x)_i^2$$

$$\frac{\partial f}{\partial q_i} = 2q_i$$

$$\nabla_q f = 2q$$

$$\nabla_W f = 2q \cdot x^T$$

$$q_1 = 0.1 \times 0.2 + 0.5 \times 0.4 = W_{1,1} x_1 + W_{1,2} x_2$$

$$0.116 = 0.44^2 + 0.52^2 = q_1^2 + q_2^2$$

$$\frac{\partial q_k}{\partial W_{i,j}} = \mathbf{1}_{k=i} x_j$$

$$\frac{\partial f}{\partial W_{i,j}} = \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial W_{i,j}}$$

$$= \sum_k (2q_k) (\mathbf{1}_{k=i} x_j)$$

$$= 2q_i x_j$$

$$\nabla_x f = 2W^T \cdot q$$

$$q_1' = 2q_1 = 2 \times 0.22 = 0.44$$

$$\frac{\partial f}{\partial W_{1,2}} = 2q_1 \times x_2 = 0.44 \times 0.4 = 0.176$$

$$\frac{\partial q_1}{\partial x_2} = W_{1,2} = 0.5$$

$$\frac{\partial f}{\partial x_2} = 2q_1 \times W_{1,2} + 2q_2 \times W_{2,2} = 0.636$$

Lecture 6 Convolutional Neural Network 卷积神经网络 CNN

1. 卷积 Convolution

Step 1: 在 Image 上移动 filter, 进行 dot product.

Convolution Layer

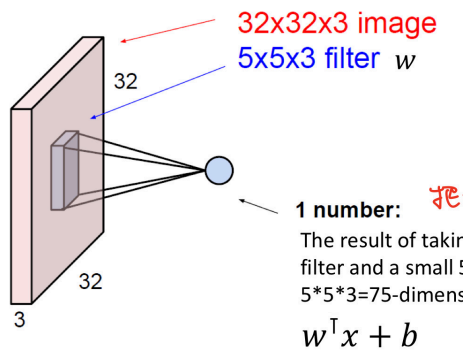


Image 为 input (粉色), 蓝色为 filter.

二者有 - 样的 depth (3), 但有不同的长和宽.

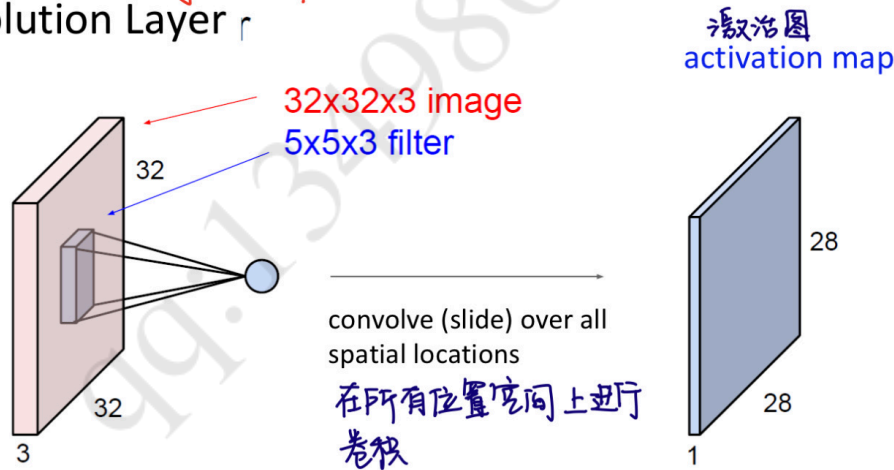
移动 filter 至 image 的每个角落进行点乘, 每次点乘得到一个 result. 即图中的蓝色圆球. 点乘的 result 为一个 number.

Step 2: 合并点乘结果 (一组 number) 得到 activation map.

filter 在 image 上一块空间进行点乘 $\rightarrow$ 获得一个 number

filter 在 image 上所有位置进行卷积 $\rightarrow$ 获得 activation map.

Convolution Layer



3

activation 的深度 = 1 (depth = 1)

在 image 上遍历 filter, 记录所有 result, 得到一个 activation map.

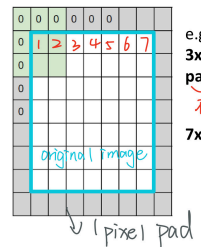
由于 filter 和 image 的 depth - 样 (均为 3), 所以 activation map 的 depth 为 1. 单个 activation map!

☆ width 和 height 问题:

① stride 步距: filter 每次移动几个像素. 默认为 1. 设为 s

② zero-padding with p pixel: 在 image 外加 n 图像素点. e.g. zero pad with 1: 在 image 外加 1 圈 0 像素点. 设为 p

通常 zero-padding with $\frac{F-1}{2}$



③ filters: 尺寸为 $F \times F$

④ Image (input): 尺寸为 $W_1 \times H_1 \times D_1$

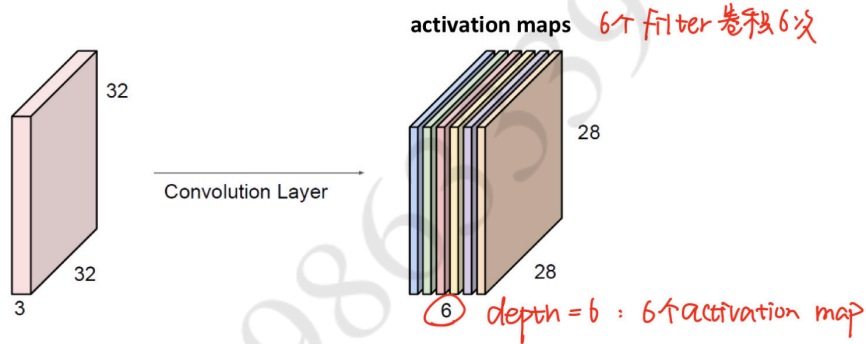
$$\text{Output (其中一边)} = \frac{W_1 - F + \text{pad} \times 2}{\text{stride}} + 1 = \frac{W_1 / H_1 - 2P}{S} + 1$$

☆ $\frac{W/D + 2P - F}{\text{stride}}$ 必须为整! Output 为 $W_2 \times H_2 \times D_2$ (W_2 和 H_2 公式如上)

$$D_2 = K = \text{num. of filters}$$

step 3: 使用不同的 filter, 获得 activation maps

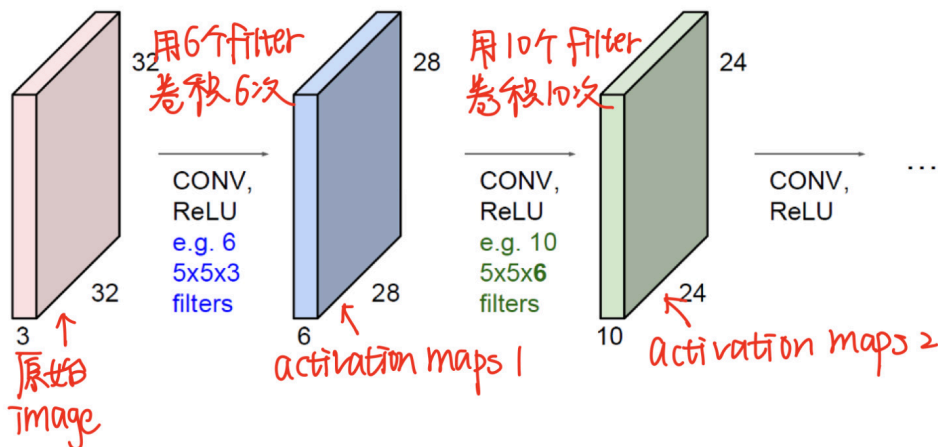
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



activation maps 的深度 depth = 卷积次数 = num. of filters used.

总体过程: Image $\xrightarrow{\text{卷积}}$ activation maps 1 $\xrightarrow{\text{卷积}}$ activation maps 2 $\rightarrow \dots$

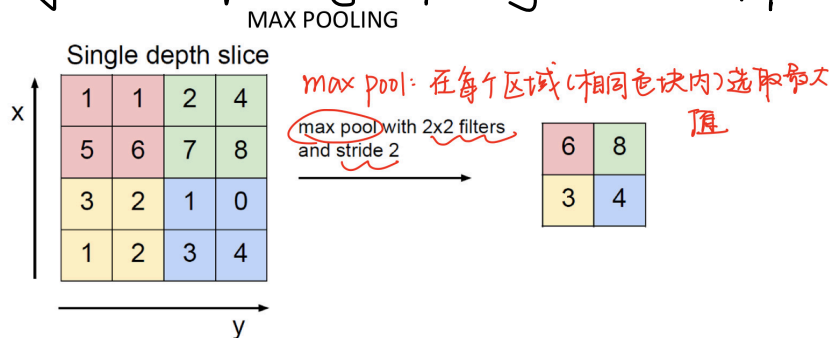
Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions



或 subsampling

2. Pooling: 在 input 的每个区域内选取特征值进行保留, 汇成一张新 map. 也需要用到 filter, 在 input 上滑动取值.

e.g.: max pooling (pooling 方法的一种)



设 input volume 的 size 为 $W_1 \times H_1 \times D_1$.

Filter 的尺寸 spatial extent 为 F

步距 stride 为 S

Pooling 没有 padding! 即无需用 0 像素进行围圈.

对于 output: $W_2 \times H_2 \times D_2$

$$① W_2 = \frac{W_1 - F}{S} + 1$$

$$② H_2 = \frac{H_1 - F}{S} + 1$$

$$③ D_2 = D_1 \text{ 卷积: } D_2 = \text{num. of filters} = k.$$

} 和卷积 input 的计算类似 (pad=0 情况)

💡 pooling vs. convolution 卷积

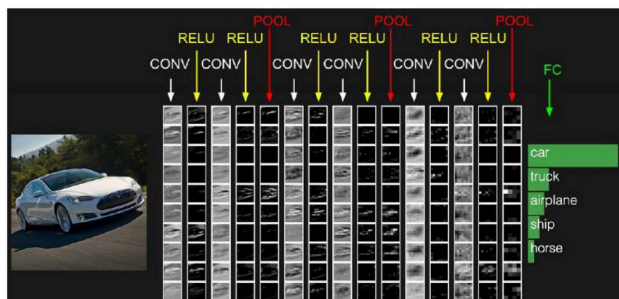
① Pooling: 利用 filter, 选取每个区域的特征值, 记成新的 map.

并不保留所有值!! Output depth $D_2 = D_1$, 深度不变

② convolution 卷积: 利用 filter 求每个区域的点积, 汇成 activation map. 所有值均参与计算. Output depth $D_2 = \text{num. of filters} = k$.

4. Fully Connected Layer (FC Layer) 全连接层

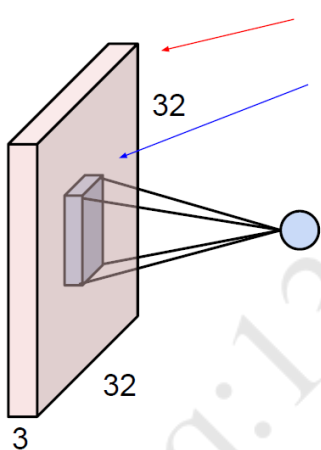
包含连接到整个 input volume 的神经, 就像普通的神经网络一样.



CNN

Convolution computation:

Convolution Layer



32x32x3 image
5x5x3 filter w

移动 filter 进行点积, 结果为一个数字
(每个匹配位置分别相乘, 再加和)

1 number:

The result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. $5*5*3=75$ -dimensional dot product + bias)

把 filter 和 image 中一块 5x5x3 的图像进行点积的结果为一个数字

$$w^T x + b$$

ReLU: $x > 0, \text{relu}(x) = x. \quad x < 0, \text{relu}(x) = 0$